

Algorithms in C++: Programming Assignment 2

AI USAGE: ASSISTIVE

You may use AI tools to learn C++, understand concepts, fix build and tooling problems, and debug your own code. You must write the algorithm yourself. See the syllabus for the full policy.

1. Objective

Your goal is to write a program that finds the lowest-risk portfolio of exactly k stocks chosen from a list of 20 stocks. The assignment practices two topics from lecture:

- **Bit operations** (Lecture 05). A portfolio is stored as a single **unsigned int**.
- **Enumeration** (Lectures 05 and 07). You will generate every k -stock portfolio in three different ways and compare the amount of work each method performs.

All three methods solve the same problem and must produce the same answer. The program checks this.

2. Background

Volatility. The volatility of a stock is the standard deviation of its returns, expressed on an annual basis. It is the standard measure of the risk of holding a stock. A stock with 30% volatility fluctuates roughly twice as much as a stock with 15% volatility.

Portfolio variance. An equally weighted portfolio of k stocks invests the same amount in each stock. Its variance (the square of its volatility) is

$$\sigma_p^2 = \frac{1}{k^2} \sum_{i \in P} \sum_{j \in P} cov[i][j]$$

where P is the set of stocks held and $cov[i][j]$ is the covariance of the returns of stocks i and j . The covariance measures how strongly two stocks move together, and $cov[i][i]$ is the variance of stock i itself. The template computes the covariance matrix from the data file.

Least volatile stocks versus least volatile portfolio. The formula sums over every pair of held stocks, so the risk of a portfolio depends on how its stocks move together and not only on the volatility of each stock. Two low-volatility stocks whose prices tend to rise and fall together provide less diversification than their individual volatilities suggest. A stock with high volatility can lower the volatility of a portfolio if it moves independently of the other holdings. As a result, the k least volatile stocks do not in general form the least volatile k -stock portfolio.

Finding the lowest-risk portfolio. No formula gives the lowest-risk portfolio of exactly k stocks, and the problem is computationally hard in general. For a small number of stocks it can be solved exactly by computing the variance of every k -stock portfolio and keeping the smallest. With 20 stocks and $k = 5$ there are 15,504 such portfolios.

3. Representing a Portfolio

The data file lists 20 stocks, one per line. Each stock is identified by its position in the file, counting from 0:

Index	Ticker	Sector	Index	Ticker	Sector
0	DUK	Utilities	10	BAC	Financials
1	SO	Utilities	11	GS	Financials
2	NEE	Utilities	12	AAPL	Technology
3	PG	Staples	13	MSFT	Technology
4	KO	Staples	14	NVDA	Technology
5	PEP	Staples	15	XOM	Energy
6	JNJ	Healthcare	16	CVX	Energy
7	MRK	Healthcare	17	COP	Energy
8	PFE	Healthcare	18	NEM	Gold
9	JPM	Financials	19	AEM	Gold

A portfolio is stored as a single 32-bit **unsigned int**. Bit i of the integer (the bit worth 2^i) is 1 if stock i is held and 0 if it is not. For example, the portfolio that holds DUK, PG, and JNJ (stocks 0, 3, and 6) is stored as follows. Bits 7 through 31 are all 0.

position	6	5	4	3	2	1	0
bit	1	0	0	1	0	0	1
stock	JNJ	PEP	KO	PG	NEE	SO	DUK

In binary this integer is 1001001, which is $64 + 8 + 1 = 73$ in decimal. Some consequences of this representation:

- The number of stocks in a portfolio equals the number of 1 bits in the integer.
- The empty portfolio is 0.
- With n stocks, every portfolio is an integer between 0 and $2^n - 1$. A 32-bit integer has room for 32 stocks; this assignment uses 20.
- The expression `1u << i` is an unsigned integer whose only 1 bit is at position i . Such a value is called a mask. Combining a portfolio with a mask using the bitwise operators `&` and `|` tests for or adds a single stock (see Lecture 05). Write `1u` rather than `1` so that the constant is unsigned.

4. Enumerating Portfolios

You will implement three methods that each return every k -stock portfolio from stocks 0 through $n - 1$, in **increasing numeric order**. For example, with $n = 4$ and $k = 2$, all three methods return the following 6 portfolios, shown in binary:

0011	0101	0110	1001	1010	1100
------	------	------	------	------	------

Method 1: Loop over every mask

Try every integer from 0 to $2^n - 1$ and keep the integers with exactly k bits set. This method is simple, but for $n = 20$ it examines 1,048,576 integers in order to keep 15,504.

Method 2: Recursion

Every portfolio of k stocks chosen from stocks 0 through $n - 1$ falls into one of two groups:

- Portfolios that do not hold stock $n - 1$. These are the portfolios returned by `get_portfolios(n - 1, k)`.
- Portfolios that hold stock $n - 1$. These are the portfolios returned by `get_portfolios(n - 1, k - 1)`, each with bit $n - 1$ set to 1.

This is the same pattern as `get_ways` in the stair climber problem (Lecture 07). Placing the portfolios from the first group before those from the second produces increasing order.

Method 3: Gosper's hack

Given a portfolio x , the following three lines compute the next larger integer with the same number of 1 bits:

```
c = x & (~x + 1);           // isolate the lowest 1 bit of x
r = x + c;                  // carry through the lowest block of 1s
next = (((r ^ x) >> 2) / c) | r;
```

Start from the smallest k -stock portfolio, $(1u \ll k) - 1$ (the k lowest bits set), and apply these steps repeatedly until the result is at least $1u \ll n$. For example, with $x = 0110$:

<code>c</code>	<code>= 0010</code>
<code>r = x + c</code>	<code>= 0110 + 0010 = 1000</code>
<code>r ^ x</code>	<code>= 1110</code>
<code>((r ^ x) >> 2) / c</code>	<code>= 0011 / 0010 = 0001</code>
<code>next</code>	<code>= 0001 1000 = 1001</code>

Written question 2 asks you to explain why each step works.

5. Problem

You are to complete a C++ program called *portfolio* in a source file called *portfolio.cpp*. The program is run as

```
./portfolio <stocks file> <number of stocks>
```

The stocks file (`stocks.csv`, provided) has one line per stock: the ticker, the sector, and then two years of daily returns (September 2024 to September 2026). The template reads this file for you. Build your program by typing `make`, which uses the provided `makefile`.

The program prints the stocks from least to most volatile, reports the number of portfolios produced by each enumeration method, checks that the three methods agree, and compares the k least volatile stocks with the lowest-risk portfolio. Stocks within a portfolio are printed in file order (bit order), not in order of volatility. Your output must match the examples below.

Case 1: No input arguments or too many arguments

```
$ ./portfolio
Usage: ./portfolio <stocks file> <number of stocks>
```

Case 2: The file cannot be opened

```
$ ./portfolio nosuchfile.csv 5
Error: Cannot open file 'nosuchfile.csv'.
```

Case 3: Invalid number of stocks

```
$ ./portfolio stocks.csv five
Error: Number of stocks must be an integer between 1 and 20.

$ ./portfolio stocks.csv 0
Error: Number of stocks must be an integer between 1 and 20.
```

The upper limit is the number of stocks in the file.

Case 4: Valid input

```
$ ./portfolio stocks.csv 5
Stocks from least to most volatile:
 1. DUK  Utilities  17.0%
 2. SO   Utilities  17.6%
 3. KO   Staples   18.0%
 4. PG   Staples   19.1%
 5. JNJ   Healthcare 19.1%
 6. PEP   Staples   21.4%
 7. PFE   Healthcare 24.0%
 8. CVX   Energy    24.3%
 9. JPM   Financials 24.7%
10. XOM   Energy    24.9%
11. BAC   Financials 25.1%
12. NEE   Utilities  25.2%
13. MSFT  Technology  28.9%
14. AAPL  Technology  28.9%
15. MRK   Healthcare 29.0%
16. COP   Energy    32.4%
17. GS    Financials  32.7%
18. AEM   Gold       42.0%
19. NVDA  Technology  44.3%
20. NEM   Gold       44.4%
```

Choosing 5 of 20 stocks:

```
Loop over all 1048576 masks: 15504 portfolios
Recursion:                  15504 portfolios
Gosper's hack:              15504 portfolios
All three lists match.
Both variance functions agree.
```

```
5 least volatile stocks: 14.0% DUK SO PG KO JNJ
Lowest-risk portfolio:  11.2% DUK PG JNJ MSFT XOM
```

6. Required Functions

The template *portfolio.cpp* reads the data file, computes the covariance matrix, and prints the results. You must implement the functions below. In each function, stock i is the stock on line i of the file, counting from 0.

Bit operations

1. `bool holds(unsigned int portfolio, int i)`
Returns true if stock i is in the portfolio, that is, if bit i is 1.
2. `int count_stocks(unsigned int portfolio)`
Returns the number of 1 bits in the portfolio. Use the method from lecture: the operation $p = p \& (p - 1)$ clears the lowest 1 bit of p , so count the number of times it can be applied before p becomes 0.
3. `vector<int> held_stocks(unsigned int portfolio)`
Returns the indices of the held stocks in increasing order. Check the lowest bit and shift right until the portfolio is 0, so that the loop stops at the highest held stock instead of always running 32 times.

Enumeration

4. `vector<unsigned int> all_masks(int n, int k)`
Method 1. Loops over every integer from 0 to $2^n - 1$ and returns those with exactly k stocks.
5. `vector<unsigned int> get_portfolios(int n, int k)`
Method 2. **This function must be recursive.** Base cases: if $k = 0$ there is exactly one portfolio, the empty portfolio $\emptyset$; if $n < k$ there are no portfolios.
6. `unsigned int next_portfolio(unsigned int x)`
One step of Gosper's hack. Returns the next larger integer with the same number of 1 bits as x .
7. `vector<unsigned int> gosper_portfolios(int n, int k)`
Method 3. Starts at $(1u \ll k) - 1$ and calls `next_portfolio` until the result is at least $1u \ll n$.

Risk

8. `double variance_all_pairs(unsigned int portfolio, const vector<vector<double>>> &cov)`
Loops over every pair (i, j) of the n stocks, adds $\text{cov}[i][j]$ when both stocks are held, and divides the sum by k^2 .
9. `double portfolio_variance(unsigned int portfolio, const vector<vector<double>>> &cov)`
Returns the same value as `variance_all_pairs`, but calls `held_stocks` first and loops only over pairs of held stocks.
10. `unsigned int lowest_risk(const vector<unsigned int> &portfolios, const vector<vector<double>>> &cov)`
Returns the portfolio with the smallest `portfolio_variance`. If two portfolios have the same variance, returns the one that appears first.

Main

Complete `main` so that it checks the command-line arguments as shown in Cases 1 to 3. The code that calls the functions above and prints the results is provided.

7. Restrictions

- Store portfolios in a plain **unsigned int**. You may not use `std::bitset`, `vector<bool>`, `__builtin_popcount`, `__builtin_ctz`, or `std::popcount`. Use the operators `<<`, `>>`, `&`, `|`, `^`, `~`, `+`, `-`, and `/`.
- Only `all_masks` may loop over every integer from 0 to $2^n - 1$.
- `get_portfolios` must be recursive, and `gospoer_portfolios` must use `next_portfolio`.

8. Tips

- Download all four files from Canvas into one folder: the template `portfolio.cpp`, `stocks.csv`, `makefile`, and `test_portfolio.sh`. The framework for the program is in the template, and you will need to fill in the functions and the argument checking in `main`. Type `make` to build and `make clean` to remove the program.
- Start with `holds` and `count_stocks`, and test them on numbers whose binary representation you have worked out by hand.
- Test each enumeration method on $(n, k) = (4, 2)$ and $(5, 3)$ before $(20, 5)$, and print the results in binary (see `get_bits` in `lectures/05/bits.cpp`). For $(4, 2)$, `next_portfolio` should step through $0011 \rightarrow 0101 \rightarrow 0110 \rightarrow 1001 \rightarrow 1010 \rightarrow 1100$.
- All three methods should return 15,504 portfolios for $(20, 5)$. If the recursive method returns the wrong number, check the base cases first.
- If the three lists do not match, find the first position at which they differ and print both values in binary.
- Be sure to comment your code and put your name and Stevens pledge at the top.
- Test your work as often as you like with the provided shell script:

```
bash test_portfolio.sh
```

It builds your program with `make`, tests each function separately, runs the finished program, and prints a score. Only the checks you fail are listed; to see every check, run it as `VERBOSE=1 bash test_portfolio.sh`.

9. Written Questions

Answer the following questions in a file called `answers.txt`.

- Counting.** Let $P(n, k)$ be the number of portfolios returned by `get_portfolios(n, k)`. Write a recurrence for $P(n, k)$, following the reasoning from Lecture 07. For $n = 20$, compute how many integers Method 1 examines for each portfolio it keeps when $k = 1$, $k = 5$, and $k = 10$. For which k is Method 1 nearly as efficient as the other two methods, and why?

2. **Gosper's hack.** Trace `next_portfolio` on 01110, showing `c`, `r`, `r ^ x`, and the result in binary. Explain what each of the three lines does. Why does dividing by `c` have the same effect as a shift?
3. **Work per portfolio.**
 - (a) Gosper's hack performs a constant amount of work per portfolio. At most how many times is a single portfolio copied into a new vector by `get_portfolios` before the top-level call returns it? Which method would you expect to be fastest for large inputs, and why?
 - (b) `variance_all_pairs` examines n^2 pairs and `portfolio_variance` examines k^2 pairs. For $n = 20$, how many pairs does each function examine when $k = 2$ and when $k = 10$?
4. **Scaling.** Suppose there were 40 stocks and you wanted every 10-stock portfolio (847,660,528 portfolios). How many integers would Method 1 examine? What in your program would fail for $n = 40$? Which of the three methods could evaluate every portfolio without storing them all in a vector, and how?
5. **Correlation.** For $k = 5$, which stocks does the lowest-risk portfolio hold that the 5 least volatile stocks do not? Where do those stocks rank in the volatility list? Explain why adding a more volatile stock can lower the volatility of a portfolio.

10. Submission and Grading

Submit a single zip file on Canvas containing exactly the following three files and no other files or folders:

- a. `portfolio.cpp`: your completed program.
- b. `answers.txt`: your answers to the written questions.
- c. `AI_USAGE.txt`: a description of any AI tools you used for this assignment and how you used them. If you did not use AI tools, state that.

The autograder tests each function separately and then runs the complete program, the same way `test_portfolio.sh` does. It also checks for your name, the Stevens pledge, and the restrictions in Section 7. Points are distributed as follows.

Component	Points
<code>holds</code> , <code>count_stocks</code>	10
<code>held_stocks</code>	5
<code>all_masks</code>	10
<code>get_portfolios</code> (recursive, correct order)	20
<code>next_portfolio</code> , <code>gosper_portfolios</code>	20
<code>variance_all_pairs</code> , <code>portfolio_variance</code>	10
<code>lowest_risk</code>	5
Argument checking in <code>main</code> (Cases 1-3)	5
Written questions	15
Total	100